



MobileFaceNet Convolutional Frontend Accelerator

ASIC (FreePDK45) and FPGA Co-Implementation

CE392 VLSI Design Projects — Final Report

Willie Liao — GEL8580

Peixin Zhang — JZH2008

Instructor: Prof. David Zaretsky

Northwestern University

June 3, 2026



Contents

Executive Summary

1	Introduction	
2	Model and Numerical Design	2
2.1	MobileFaceNet Topology	2
2.2	Quantization	3
3	Broader Considerations	3
4	Design Constraints and Requirements	4
4.1	Process and IP Availability	4
4.2	Verification Capability	4
5	Hardware Architecture	5
5.1	Architecture Overview	5
5.2	Frontend Design (RTL Architecture)	5
5.2.1	Top-Level NPU and Memory Hierarchy	5
5.2.2	Operator Engines	7
5.2.3	Design Rationale	7
5.3	Backend Design (Physical Implementation)	9
5.3.1	ASIC Backend: Genus Synthesis and Innovus P&R	9
5.3.2	FPGA Backend: Vivado on Zynq UltraScale+	13
6	Design Optimizations	14
6.1	ASIC Optimization Journey	14
6.2	FPGA Optimization	16
7	Engineering Standards	16
8	Testing / Verification	17
8.1	Verification Flow	17
8.2	ASIC Verification: Three-Tier + SDF Signoff	19
8.3	FPGA Verification: Zynq On-Board Readback	19
9	Ethical Considerations	20
10	Conclusions	21

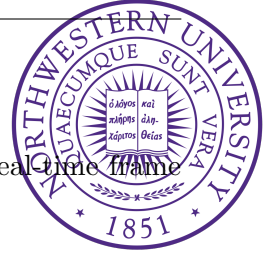


List of Figures

1	MobileFaceNet topology overview.	2
2	NPU top-level architecture.	6
3	1×1 Conv engine and slide direction.	7
4	3×3 Layer engine and direction.	7
5	Why systolic array.	8
6	Innovus post-route layout.	10
7	Clock-tree synthesis result.	11
8	Post-route timing and clock-domain reports.	12
9	Routing and area utilisation.	12
10	Innovus post-route power report.	13
11	FPGA block design (Vivado IP integrator).	13
12	Post-route floorplan evolution v1 $\rightarrow$ v2 $\rightarrow$ v3.	14
13	Macro placement evolution v2 $\rightarrow$ v3.	15
14	QuantFace face-verification benchmark suite.	18
15	FPGA on-board embedding readback vs reference.	20

List of Tables

1	MobileFaceNet Layer Configuration ($3 \times 112 \times 112$ input)	3
2	Quantized Hardware Datatypes	3
3	Design Constraints and Requirements	4
4	Implementation Comparison: FPGA and ASIC Tracks	5
5	NPU Global Memory Hierarchy	6
6	Genus Synthesis Results	9
7	Innovus Post-Route Results	9
8	FPGA Resource Utilisation (Vivado Implementation)	13
9	FPGA Throughput Comparison (one inference, 112×96 image)	14
10	ASIC Architecture Evolution: v1 $\rightarrow$ v2 $\rightarrow$ v3	15
11	v3 Cumulative Micro-Optimizations	16
12	Engineering Standards Applied	17
13	Hardware-Spec Fixed-Point vs QuantFace PyTorch Reference	18
14	Verification Coverage Matrix	18
15	ASIC Verification Results	19
16	Gate-Level Simulation Wall-Clock Comparison	19
17	Abuse Vectors and Mitigations	21
18	Final Metrics vs Design Constraints	22
19	Suggested Follow-on Work	22



Executive Summary

Problem. Edge face verification needs a 50-layer CNN to run under ~ 100 mW at real-time frame rates. General-purpose CPUs and GPUs are too power-hungry to fit that envelope.

Approach. We designed a hybrid INT8 CNN accelerator specialised for MobileFaceNet. The compute fabric combines a 14×16 systolic array for the 1×1 convolutions that dominate the workload, a 3×3 standard array for 3×3 convolutions and 3×3 depthwise convolutions, and a dedicated 7×7 depthwise engine. The same RTL is brought up along two complementary tracks: a Zynq UltraScale+ FPGA bitstream for hardware-in-the-loop verification, and a FreePDK45 Innovus place-and-route flow for ASIC area, power, and timing data.

Key Results.

- **FPGA (Zynq UltraScale+):** 150 MHz, 19.18 M cycles per inference, 127.8 ms per image, $\times 7.18$ over an i7-9700 baseline. Resource utilisation 25 % LUT, 20 % FF, 54 % BRAM, 10 % DSP.
- **ASIC (FreePDK45):** 200 MHz, 19.18 M cycles per inference, 95.88 ms per image, $\times 9.57$ over the same i7-9700 baseline. Core area $720 \times 720 \mu\text{m}^2$, total power 96.32 mW, energy $9.24 \mu\text{J}$ per inference, DRC/connectivity/antenna 0.
- **Functional verification:** bit-exact 50/50 layer match against the C-model at RTL, post-syn netlist, post-route netlist, and SDF-annotated post-route gate level (zero timing violations).
- **Model accuracy:** hardware-spec fixed-point matches the QuantFace INT8 reference within 0.20 % on LFW and -0.12 % on AgeDB-30, both inside the protocol's statistical error.

Limitations.

- No commercial formal LEC tool was available; equivalence is shown by full 50-layer gate-level simulation rather than mathematical proof.
- Single-corner timing signoff (typical PVT only); no SI/CRPR analysis.
- ASIC track is design-kit physical implementation only, not taped out. The FPGA bitstream targets a development board, not a productised platform.

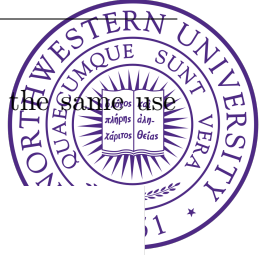
1 Introduction

Goal. This project builds an INT8 hybrid CNN accelerator that runs the full 50-layer MobileFaceNet inference flow on-device, producing a 128-dimensional face embedding with no host intervention between layers. The accelerator targets edge use cases (mobile face unlock, on-device door access, in-camera identification) where the operating envelope is sub-100 mW at real-time frame rates.

Approach overview. Rather than building a fully programmable NPU, we specialise the datapath around MobileFaceNet's actual operator mix. The accelerator contains three operator engines behind a shared Switcher: a 14×16 systolic array for 1×1 convolutions (which account for ~ 92 % of the network's compute), a 3×3 standard array for 3×3 standard and depthwise convolutions, and a dedicated 7×7 depthwise engine for the global pooling layer. All weights, activations, and outputs are INT8, with the model trained via the QuantFace flow [1].

The same RTL is brought up along two implementation tracks. The Zynq UltraScale+ FPGA bitstream provides hardware-in-the-loop verification of the full PS-PL data path; the FreePDK45 Innovus flow provides ASIC-level area, power, and timing data with full-flow signoff (Genus synthesis, Innovus place-and-route, SDF-annotated gate-level simulation). Both tracks execute the same 19.18 M cycles per inference and have been verified bit-exact against the same C-model golden reference.

Prior art. The MobileFaceNet topology is from Chen et al. [2], with the non-square 112×96 input variant from Sun et al. [5]. The INT8 quantisation flow and the face-verification benchmark suite (LFW, CFP, AgeDB, CALFW, CPLFW, IJB-B/C) follow QuantFace [1]. The systolic-array dataflow



draws on Eyeriss [3]; the closely related edge-deployment model EdgeFace [4] targets the same use case at the model level, whereas our contribution is the hardware track.

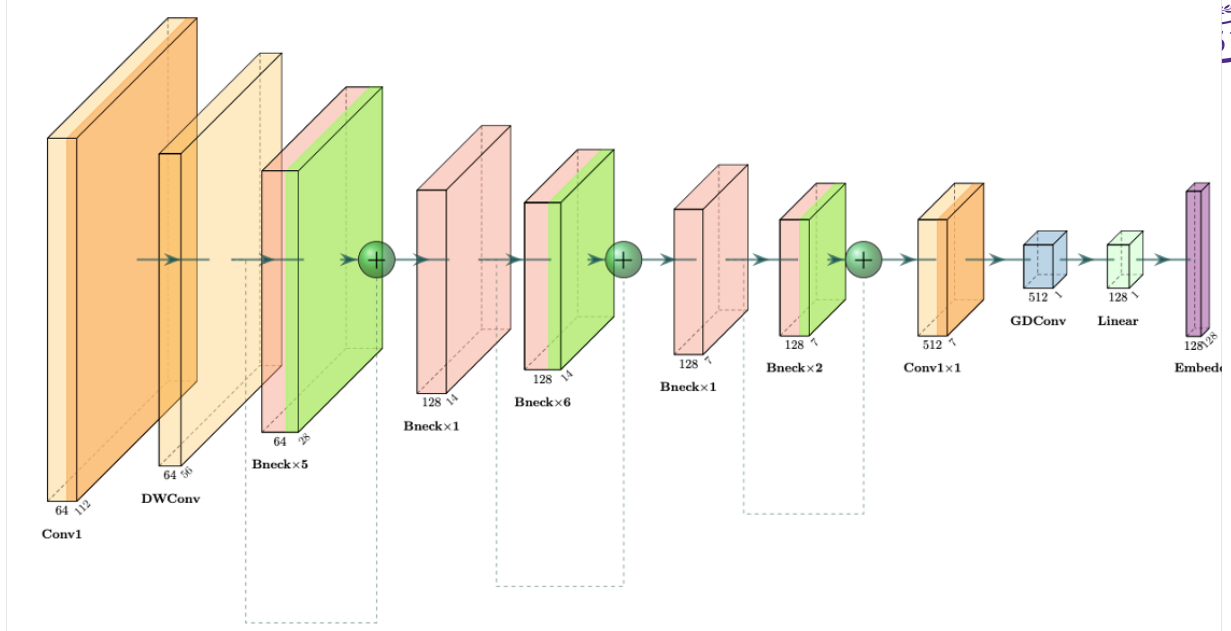


Figure 1: MobileFaceNet topology overview: 50 layers from the initial 3×3 convolution through five bottleneck stages to the final 7×6 global depthwise convolution and 1×1 linear projection producing a 128-d embedding.

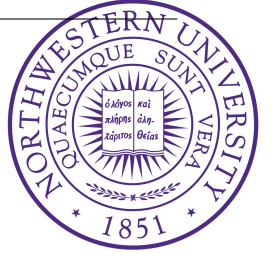
Report structure. Section 2 fixes the model topology and INT8 quantisation. Section 3 discusses broader impacts and Section 4 states design constraints. Section 5 describes the hardware architecture (shared frontend and the two backend tracks). Section 6 walks through the architectural optimisation journey. Section 7 lists engineering standards. Section 8 presents functional and timing verification. Section 9 addresses ethical considerations specific to face recognition. Section 10 concludes.

2 Model and Numerical Design

Before any hardware is built, the model topology and the numerical format must be fixed. Both decisions are constraints that the hardware datapath is then specialised around.

2.1 MobileFaceNet Topology

We target the original MobileFaceNet topology [2] with a square $3 \times 112 \times 112$ input and a 7×7 global depthwise convolution, as also adopted by the Eyeriss-style accelerator in [3]. The complete layer schedule is shown in Table 1. When each Bottleneck is expanded into its three constituent sub-layers (PW1, DW, PW2), the full model comprises **50 layers** (L0–L49).

Table 1: MobileFaceNet Layer Configuration ($3 \times 112 \times 112$ input)

Input shape	Operator	t	c	n	s
$3 \times 112 \times 112$	Conv 3×3	–	64	1	2
$64 \times 56 \times 56$	DW Conv 3×3	–	64	1	1
$64 \times 56 \times 56$	Bottleneck	2	64	5	2
$64 \times 28 \times 28$	Bottleneck	4	128	1	2
$128 \times 14 \times 14$	Bottleneck	2	128	6	1
$128 \times 14 \times 14$	Bottleneck	4	128	1	2
$128 \times 7 \times 7$	Bottleneck	2	128	2	1
$128 \times 7 \times 7$	Conv 1×1	–	512	1	1
$512 \times 7 \times 7$	GDCConv 7×7	–	512	1	1
$512 \times 1 \times 1$	Linear (PW 1×1)	–	128	1	1

The recurring spatial dimensions across the network are $H = W \in \{7, 14, 28, 56\}$ and the channel dimensions are $C_{\text{in}}, C_{\text{out}} \in \{64, 128, 256, 512\}$. These ranges drive the array-size choice in Section 5.2.

2.2 Quantization

The reference model is trained in INT8 via the QuantFace flow [1] and the hardware mirrors that representation directly. All weights, activations, and partial sums use 8-bit fixed-point storage, with 32-bit accumulators inside each MAC. Weights are stored with a symmetric scheme ($zp_w = 0$) and activations with an asymmetric scheme (non-zero zp_a). The MAC operation expands as

$$(act - zp_a)(wgt - zp_w) = act \cdot wgt - zp_w \cdot act - zp_a \cdot wgt + zp_a \cdot zp_w. \quad (1)$$

Because $zp_w = 0$ for symmetric weights, the last two terms in Eq. (1) vanish; the runtime cost of asymmetric activation reduces to one $-zp_a \cdot wgt$ correction per output channel, which is absorbed into the bias and applied once per layer. The final hardware bit-widths are summarised in Table 2.

Table 2: Quantized Hardware Datatypes

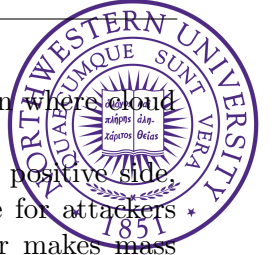
Quantity	Scheme	Width
Activation	asymmetric (non-zero zp_a)	int8
Weight	symmetric ($zp_w = 0$)	int8
MAC accumulator	signed	int32
Bias	signed, pre-folded with $-zp_a \cdot \sum wgt$	int32
PReLU α	signed	int8

The accuracy of this INT8 representation against the floating-point QuantFace baseline is presented in Section 8.1, alongside the broader face-verification benchmark suite.

3 Broader Considerations

Face verification is one of the most widely deployed biometric modalities, shipping in hundreds of millions of units a year through smartphones, smart locks, doorbell cameras, point-of-sale terminals, and access gates. Almost all of those products use edge inference: the embedding network runs on the device, and only the 128-dimensional vector or a match decision leaves the chip. An accelerator that fits the sub-100mW envelope of a battery-powered device at real-time frame rates is a direct commercial fit for that segment.

The clearest integration target is as a small NPU sub-system inside a larger SoC, sharing the host AXI bus and memory hierarchy. Concrete uses we considered were mobile face unlock with an always-on camera, smart doorbells and cameras that recognise locally instead of streaming to



the cloud, badge-less building access running offline, and in-vehicle driver identification where cloud round-trip latency is unacceptable.

Pushing face recognition to the edge has both privacy benefits and risks. On the positive side, raw images and templates never leave the device, so there is no centralised database for attackers or governments to compel. On the negative side, the same drop in cost and power makes mass deployment in surveillance contexts much easier; the same chip that unlocks a private user's phone can identify a stranger on a public street. The technology is dual-use, and deployment policy decides which outcome dominates. Specific safeguards are in Section 9.

Several legal frameworks already constrain how this kind of accelerator can be productised. Face data is special-category personal data under EU GDPR Article 9, with similar provisions in California's CCPA and other jurisdictions. The EU AI Act (Articles 5 and 6) classifies real-time remote biometric identification in public spaces as either prohibited or high-risk depending on context. A commercial product built on this accelerator therefore needs consent capture, purpose limitation, on-device template storage, and a documented opt-out path. These obligations shape the chip interface in concrete ways: only the 128-d embedding is exposed off-die, the raw image stays inside, and in a production SoC the embedding storage would sit inside a hardware-isolated enclave.

4 Design Constraints and Requirements

Table 3 summarises the hard and soft constraints that drove the architectural and implementation decisions.

Table 3: Design Constraints and Requirements

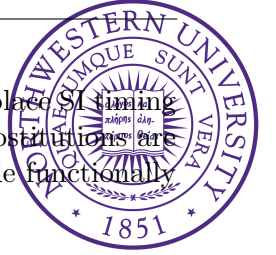
Category	Constraint / Requirement	Type
Functional	Execute 50 MobileFaceNet layers autonomously	Hard
Functional	Produce 128-d float-equivalent embedding within <0.001 of C-model	Hard
Numerics	INT8 weights and activations (w8a8), INT32 accumulators	Hard
Process (ASIC)	FreePDK45 standard-cell library (NCSU PDK)	Hard
Memory IP	No commercial SRAM IP; must use OpenRAM open-source generator	Hard
Clock target	≥ 100 MHz baseline; 200 MHz stretch goal	Soft
Power budget	< 100 mW (edge battery-powered use case)	Soft
Area target	$< 1 \text{ mm}^2$ core area (educational-PDK norm)	Soft
DRC / antenna	Zero violations at signoff	Hard
Timing signoff	Setup/hold WNS > 0 at all corners run	Hard
Verification	50/50 layer bit-exact match against C-model	Hard
Toolchain	Cadence Genus (synthesis), Innovus (P&R), Xcelium (simulation)	Hard
FPGA target		Hard

4.1 Process and IP Availability

The FreePDK45 educational PDK provides only standard-cell timing and LEF/Liberty data; no foundry SRAM compiler is included. We therefore generated all on-chip SRAM macros via the OpenRAM open-source SRAM compiler, which produces synthesizable behavioural models for simulation and LEF/Liberty/GDS views for physical implementation. This constraint shaped the memory subsystem: we use only two SRAM macros (activation buffer and partial-sum buffer) and keep small constant storage (weights, biases, PReLU α , layer configuration) as combinational ROMs that the synthesizer maps to standard-cell mux trees.

4.2 Verification Capability

The lab environment does not provide a commercial formal-equivalence checker (Conformal LEC / Formality), nor signal-integrity-aware static timing (PrimeTime SI). We therefore replace formal



LEC with full 50-layer gate-level simulation against the C-model golden vectors, and replace SI timing with SDF-annotated gate-level simulation using the Innovus-extracted SDF. These substitutions are slower than the commercial flows (a single full SDF run takes several days) but provide functionally equivalent coverage for non-tape-out signoff.

5 Hardware Architecture

5.1 Architecture Overview

The accelerator is a hybrid CNN NPU specialised for MobileFaceNet face verification. It supports dynamically-sized 1×1 convolution, 3×3 convolution, 3×3 and 7×7 depthwise convolution, residual addition, and PReLU activation. Inputs and outputs use INT8 quantisation throughout. The same RTL is brought up along two implementation paths sharing identical functional behaviour: a Zynq UltraScale+ FPGA bitstream used for on-board verification and rapid iteration, and an Innovus place-and-route in FreePDK45 used to obtain ASIC-level area, power, and timing data. Table 4 summarises the two paths against the shared design.

Table 4: Implementation Comparison: FPGA and ASIC Tracks

	FPGA track	ASIC track
Target	Zynq UltraScale+ MPSoC	FreePDK45 standard cells
Compute fabric	14×16 SA + 3×3 array + dw 7×7 (shared RTL)	same RTL, retargeted
Activation memory	BRAM (Vivado-inferred)	OpenRAM SRAM macro
Weight memory	BRAM (1 MB)	combinational ROM (no usable SRAM compiler entry)
Param / config RAM	BRAM	BRAM-style synthesised RAM
Achieved clock	150 MHz	200 MHz
Cycles / inference	19,175,557	19,175,557
Latency / image	127.8 ms	95.88 ms
Speedup vs i7-9700 (917.37 ms)	$\times 7.18$	$\times 9.57$
Implementation tool	Vivado	Cadence Genus + Innovus
Verification	on-board PS readback	RTL + Genus syn + Innovus post-route + SDF

The two tracks are complementary. The FPGA bitstream provides hardware-in-the-loop functional validation at the system level, with the Zynq Processing System (PS) driving the NPU and reading back the 128-d embedding through AXI. The ASIC flow quantifies physical metrics (area, power, timing closure) on a standard-cell technology, with full-flow signoff via Genus synthesis, Innovus place-and-route, and SDF-annotated gate-level simulation.

5.2 Frontend Design (RTL Architecture)

The frontend design covers the register-transfer-level architecture shared by both implementation paths: the compute datapath, the memory hierarchy at the RTL level, and the controller finite-state machine. ASIC-specific and FPGA-specific RTL details follow in the two sub-sub-sections below.

5.2.1 Top-Level NPU and Memory Hierarchy

Figure 2 shows the top-level NPU. Three operator engines sit behind a shared Switcher that arbitrates access to the global memories and routes the layer-config descriptor into the active engine.

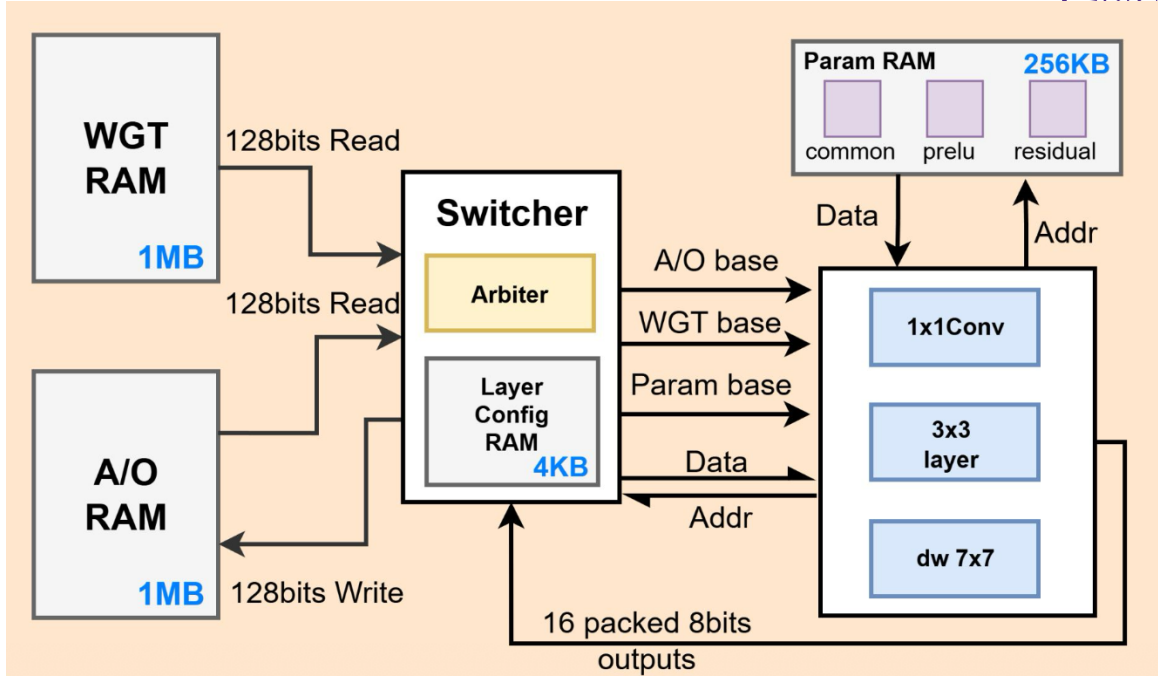


Figure 2: NPU top level. The Switcher routes the active layer descriptor and data between the global memories (WGT RAM, A/O RAM, Param RAM, Layer Config RAM) and one of three operator engines: 1×1 Conv, 3×3 Layer, or dw 7×7 .

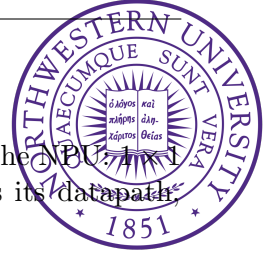
Global memories. Table 5 lists the four shared memories. Sizes are the FPGA-side BRAM allocations; the ASIC track keeps the same depth but realises the largest memories with different physical resources, as described in Section 5.3.1.

Table 5: NPU Global Memory Hierarchy

Memory	Size	Contents
WGT RAM	1 MB	Quantised INT8 weights for all 50 layers
A/O RAM	1 MB	Input activations and intermediate/final outputs
Param RAM	256 KB	Per-layer common params + PReLU α + residual offsets
Layer Config RAM	4 KB	64-bit layer descriptors driving the Switcher

Switcher and execution flow. On each layer the Switcher reads the next 64-bit descriptor from the Layer Config RAM, decodes the operator type and the WGT / A/O / Param base addresses, and selects the matching operator engine. The PS-side host needs only to write the start address and assert a start signal: the Switcher walks through the 50-entry descriptor table and stops when the last layer completes, at which point the 128-d embedding has been written back into the A/O RAM ready for AXI readback.

Compute datapath summary. The three engines reuse a common Window Generator $\rightarrow$ array $\rightarrow$ Post-Process pipeline. The Window Generator slices A/O RAM into per-cycle patches; the array (sized to the operator) does the multiply-accumulate; the Post-Process unit applies bias addition, requantisation back to INT8, residual add, and PReLU before packing 16 outputs into a 128-bit write back into A/O RAM. Detailed datapaths for the 1×1 Conv and 3×3 Layer engines follow in Section 5.2.2; the design rationale (Why systolic array, why 14×16 , quantisation scheme) is presented in Section 5.2.3.



5.2.2 Operator Engines

1×1 Conv engine (14×16 systolic array). The 1×1 engine is the workhorse of the MBU. 1×1 convolution accounts for roughly 92% of MobileFaceNet's compute. Figure 3a shows its datapath and Figure 3b shows the spatial-tile sliding direction across the input feature map.

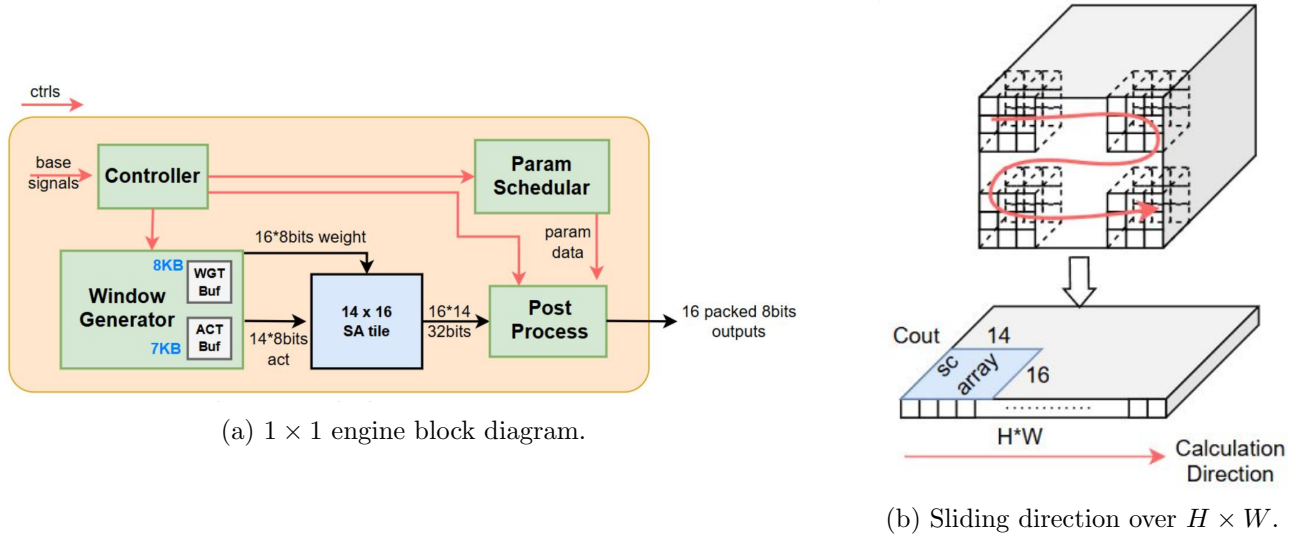


Figure 3: 1×1 Conv engine: 14×16 SA tile fed by a Window Generator with 8KB WGT Buf and 7KB ACT Buf, outputting 16*14 32-bit partial sums per cycle into the Post-Process unit.

The Controller drives the Window Generator to load 14 INT8 activations per cycle (one per SA row) and 16 INT8 weights per cycle (one per SA column). The $14 \times 16 = 224$ -PE tile produces 224 32-bit partial sums per cycle. The Param Scheduler streams bias, requantisation factors, and PReLU α into the Post-Process unit, which packs 16 INT8 outputs into the A/O RAM each cycle.

3×3 Layer engine. The 3×3 engine handles both standard 3×3 convolution and 3×3 depthwise convolution. Its datapath is shown in Figure 4a, and the calculation direction (through the input volume) in Figure 4b.

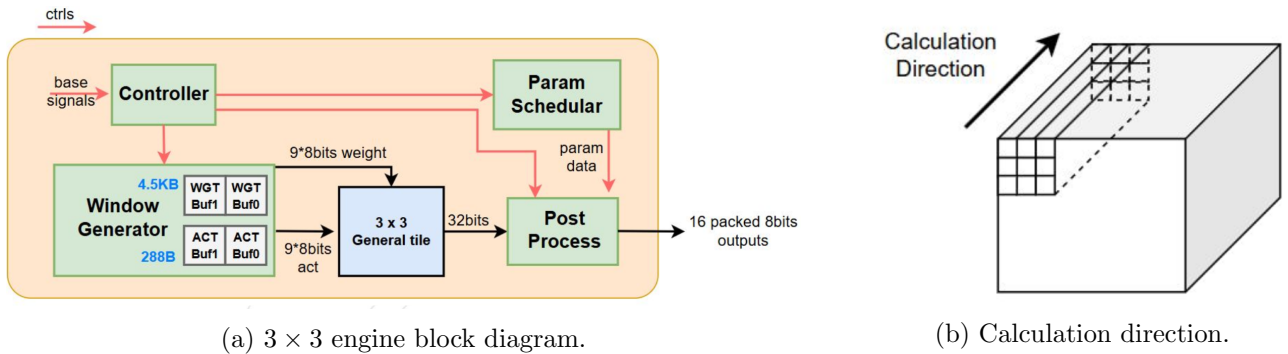
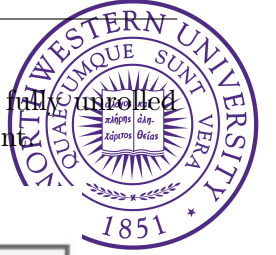


Figure 4: 3×3 Layer engine: a 3×3 general tile fed by a 4.5 KB WGT buf and a 288 B ACT buf, with double-buffering (Buf0/Buf1) so the next window is fetched while the current one computes. Outputs 16 packed 8-bit values per cycle.

5.2.3 Design Rationale

Why a systolic array. 1×1 convolution dominates the network's compute budget (around 92%), so the 1×1 engine drives most of the area and energy budget. A systolic-array organisation amortises the memory-bandwidth pressure of the dense matrix-vector multiply: the SA tile reuses each fetched weight across 14 spatial positions and each fetched activation across 16 output channels, so per cycle



the engine needs only $14 + 16 = 30$ INT8 reads, against the $14 \times 16 \cdot 2 = 448$ that a fully unrolled MAC array would require. Figure 5 illustrates the resulting weight-reuse data movement.

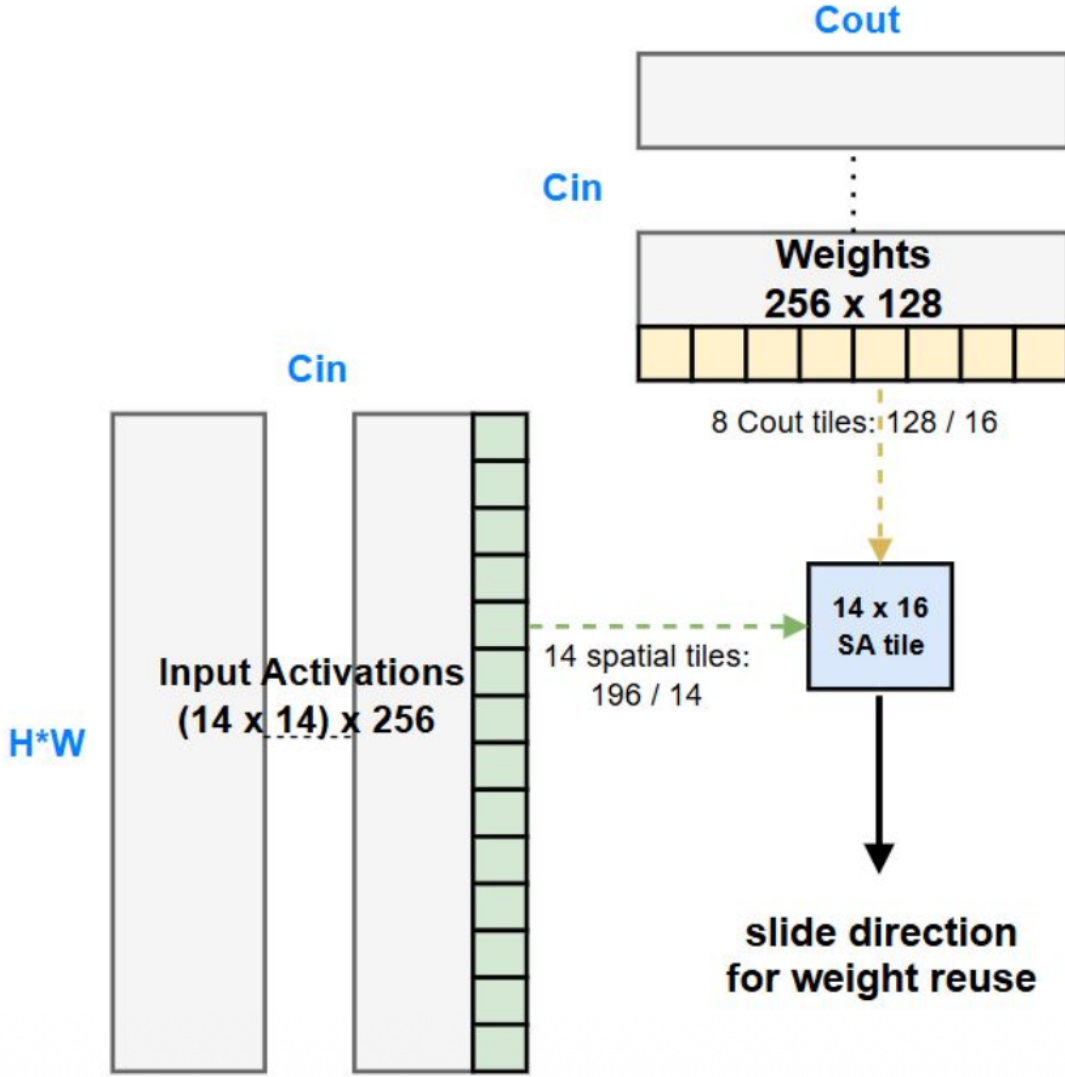
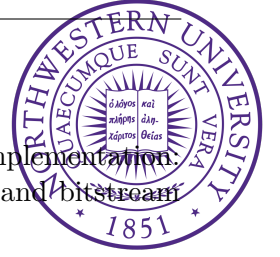


Figure 5: Weight-stationary slide direction in the 14×16 SA tile: each weight block is held in the array and reused across spatial tiles before the next slide.

Why 14×16 . MobileFaceNet’s recurring dimensions are $H, W \in \{7, 14, 28, 56\}$ and $C_{in}, C_{out} \in \{64, 128, 256, 512\}$. Setting the SA spatial dimension to 14 lets the engine cover an entire row of a $14 \times W$ feature map in one tile (and partially tile rows from 28 or 56), and setting the output dimension to 16 divides every C_{out} value cleanly. With this choice the layer scheduler never needs to handle awkward residual tiles. A larger SA (e.g. 28 or 32 wide) would idle PEs on the smallest $7 \times W$ layers; a smaller one would waste reuse on the larger layers.

Quantization scheme. Activations carry a non-zero zero-point (zp_a) and weights are kept symmetric ($zp_w = 0$), so the asymmetric-MAC expansion of Eq. (1) collapses to a single $-zp_a \cdot \sum wgt$ correction per output channel, applied off the critical path as part of the bias.

Why not a single oversized array. A traditional unified 3×3 array was considered for all operators. Two issues made it unattractive: it complicates the outer controller because 1×1 and 3×3 have very different sliding patterns, and the effective C_{in} available to the 3×3 array per cycle is only $3 \times 3 \times 3 = 27$ for the first layer and $3 \times 3 = 9$ for depthwise layers, which under-utilises a large array. Splitting the operators across two specialised engines lets each engine match its workload.



5.3 Backend Design (Physical Implementation)

The backend design covers the technology-specific mapping of the RTL to a physical implementation. It includes standard-cell synthesis and place-and-route for the ASIC path, and FPGA synthesis and bitstream generation for the FPGA path.

5.3.1 ASIC Backend: Genus Synthesis and Innovus P&R

The RTL is synthesised with Cadence Genus against the FreePDK45 standard-cell library. Three on-chip memories from the top-level diagram (Figure 2) are retargeted for the ASIC: the A/O RAM is realised as an OpenRAM SRAM macro to keep activation accesses dense and low-power; the partial-sum staging register file inside the Post-Process unit is realised as a second OpenRAM SRAM macro; the WGT RAM remains a combinational ROM synthesised from the INT8 weight constants, because FreePDK45 does not ship an SRAM compiler entry sized for the 1 MB weight memory in this PDK. The Param RAM and Layer Config RAM are small enough to synthesise into standard cells. Synthesis runs at a 5.0 ns target period. Results are in Table 6.

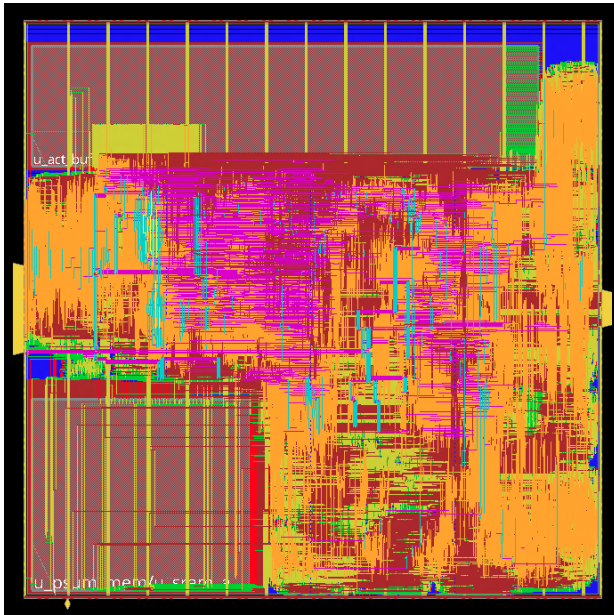
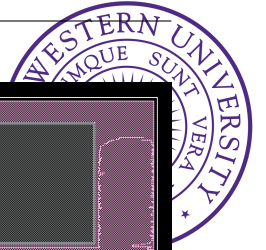
Table 6: Genus Synthesis Results

Metric	Value
Target clock period	5.0 ns (200 MHz)
Worst slack (WNS)	+0.177 ns
Total cells	88,018
Total area (cells only)	334,956 μm^2
SRAM macros	2 (A/O buffer 97 K μm^2 + partial-sum 66 K μm^2)
Critical path	A/O buffer read $\rightarrow$ SA tile multiplier $\rightarrow$ adder tree $\rightarrow$ Post-Process register

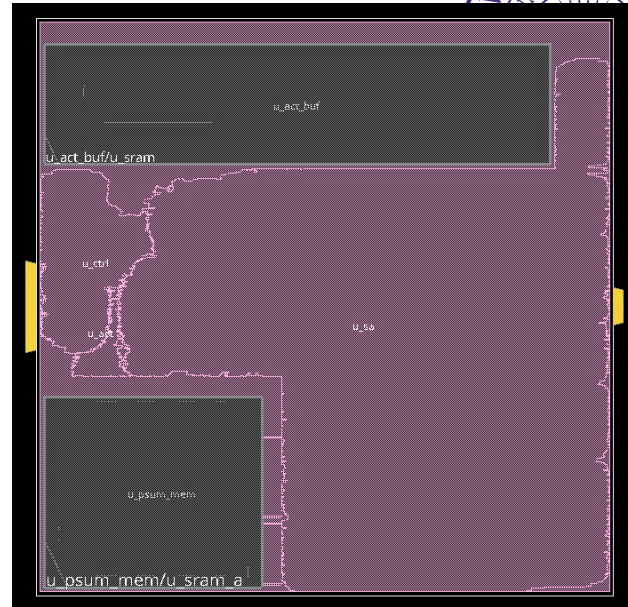
The netlist is then placed and routed in Cadence Innovus on a $720 \times 720 \mu\text{m}^2$ floorplan, shrunk from an initial 800×700 once timing closed. The two SRAM macros are placed manually so the SA dataflow stays compact, and the clock tree uses non-default routing (CTS_2W1S on leaf nets, CTS_2W2S on trunks with VSS shielding) to hold skew under 70 ps. Before LEF import, `fix_sram.lef.py` strips the over-conservative M3/M4 obstructions from the OpenRAM LEF and snaps off-grid pin coordinates to the $0.005 \mu\text{m}$ manufacturing grid; without this step the SRAM dout pins on M3 have no legal via3/M4 escape and the router produces ~ 480 DRC SHORT/SPACING violations. The final post-route metrics are in Table 7 and the layout in Figure 6.

Table 7: Innovus Post-Route Results

Category	Result
Core floorplan	$720 \times 720 \mu\text{m}^2$ (518,400 μm^2)
Placement density	59.5 %
Setup WNS	+0.089 ns (MET)
Hold WNS	+0.437 ns (MET)
Setup / Hold TNS	0 / 0
DRC violations	0
Connectivity	no problems or warnings
Antenna	no violations
Total power	96.32 mW (Switching 43.76, Internal 24.6, Leakage 3.0, Macro 24.9)
Cycles per inference	19,175,557
Throughput	10.43 FPS @ 200 MHz
Energy per inference	9.24 μJ



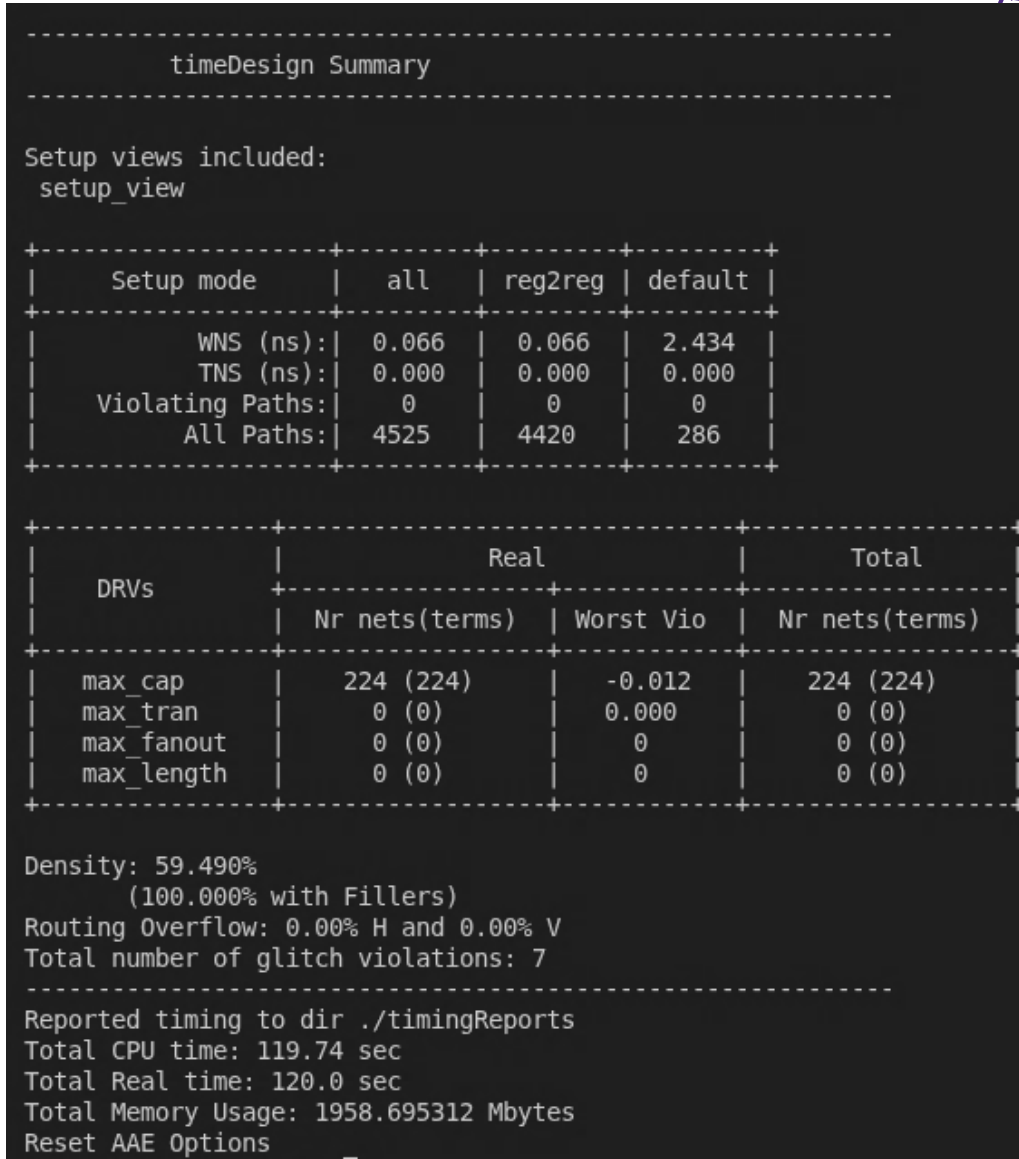
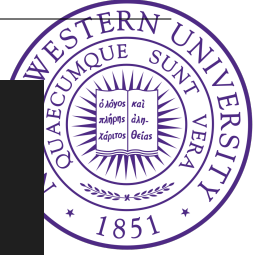
(a) Full die view ($720 \times 720 \mu\text{m}^2$).



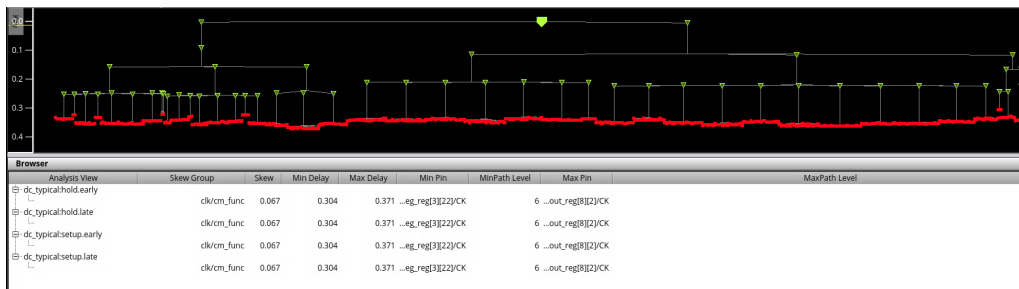
(b) Detail view: SA + SRAM macros + controller cells.

Figure 6: Final Innovus post-route layout. Two OpenRAM SRAM macros (A/O buffer left, partial-sum buffer right) bracket the central SA placement.

The CTS report (Figure 7) shows balanced clock arrival times across the die with insertion-delay $\mu = 0.348 \text{ ns}$, $\sigma = 0.009 \text{ ns}$ and skew 0.067 ns .



(a) Post-CTS placement and clock buffers.



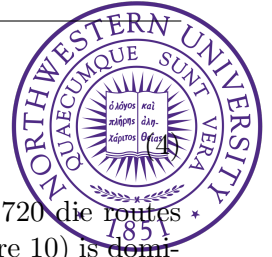
(b) Clock-tree topology and balanced fanout.

Figure 7: Clock-tree synthesis result. Skew 0.067 ns, insertion-delay $\mu = 0.348$ ns, $\sigma = 0.009$ ns.

With WNS = +0.089 ns on the 5.0 ns target, the critical path measures $T_{\text{crit}} = 4.911$ ns. The achievable frequency and the resulting throughput and energy are then

$$F_{\text{max}} = \frac{1}{T_{\text{crit}}} = \frac{1}{4.911 \text{ ns}} \approx 203.6 \text{ MHz}, \quad (2)$$

$$\text{FPS} = \frac{F_{\text{clk}}}{C_{\text{inf}}} = \frac{200,000,000}{19,175,557} \approx 10.43 \text{ frames/s}, \quad (3)$$



$$T_{\text{inf}} = \frac{C_{\text{inf}}}{F_{\text{clk}}} \approx 95.88 \text{ ms}, \quad E_{\text{inf}} = P_{\text{total}} \cdot T_{\text{inf}} \approx 9.24 \mu\text{J}.$$

The Innovus reports (Figures 8 and 9) confirm these numbers and show the 720×720 die routes without congestion hotspots at 59.5% placement density. The power breakdown (Figure 10) is dominated by switching activity in the SA, as expected when the 1×1 engine fires every cycle at 200 MHz.

```

innovus 6> report_timing
#####
# Generated by: Cadence Innovus 16.20-p002.1
# OS: Linux x86_64 (Host ID moore.wot.ece.northwestern.edu)
# Generated on: Tue May 26 23:10:42 2026
# Design: mfn_frontend_top_v3
# Command: report_timing
#####
Path 1: MET Setup Check with Pin u_sa/row_dot_reg_reg[8][39]/CK
Endpoint: u_sa/row_dot_reg_reg[8][39]/SI (^) checked with leading edge of
'clk'
Beginpoint: u_act_buf/act_out_reg[0][0]/Q (^) triggered by leading edge of
'clk'
Path Groups: {clk}
Analysis View: setup_view
Other End Arrival Time 0.427
- Setup 0.066
+ Phase Shift 5.000
- Uncertainty 0.100
= Required Time 5.262
- Arrival Time 5.172
= Slack Time 0.089
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) -0.033
= Beginpoint Arrival Time -0.033

```

(a) Innovus report_timing: setup WNS +0.089 ns.

```

innovus 7> report_clocks
#####
# Generated by: Cadence Innovus 16.20-p002.1
# OS: Linux x86_64 (Host ID moore.wot.ece.northwestern.edu)
# Generated on: Tue May 26 23:10:48 2026
# Design: mfn_frontend_top_v3
# Command: report_clocks
#####

```

Clock Descriptions							Attributes	
Clock Name	Source	View	Period	Lead	Trail	Generated	Propagated	
clk	clk	setup_view	5.000	0.000	2.500	n	n	

(b) Per-clock summary: 200 MHz domain.

Figure 8: Post-route timing and clock-domain reports.

```

-----
timeDesign Summary
-----
Setup views included:
setup_view

```

Setup mode	all	reg2reg	default
WNS (ns):	0.089	0.089	2.394
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	4525	4420	286

DRVs	Real		Total
	Nr nets (terms)	Worst Vio	Nr nets (terms)
max_cap	222 (222)	-0.012	222 (222)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 59.490%
 (100.000% with Fillers)
 Total number of glitch violations: 7

 Reported timing to dir ./timingReports
 Total CPU time: 74.91 sec
 Total Real time: 75.0 sec
 Total Memory Usage: 2024.429688 Mbytes
 Reset AAE Options

(a) Post-route view (optsRoute).

```

innovus 4> report_area

```

Depth	Name	#Inst	Area (um^2)
0	mfn_frontend_top_v3	90360	359740.49105
1	u_psum_mem	85	66448.993
1	u_act	812	2249.296
1	u_sa	79630	173958.68
1	u_ctrl	7994	16799.496
1	u_bias_rom	0	0.0
1	u_act_buf	1642	100058.72405
1	u_wgt_rom	0	0.0
1	u_prelu_rom	0	0.0
1	u_cfg_rom	0	0.0

(b) Area / utilisation report.

Figure 9: Routing and area utilisation: zero DRC violations, 59.5% density.



Total Power		

Total Internal Power:	48.60088682	50.4594%
Total Switching Power:	43.75658635	45.4298%
Total Leakage Power:	3.95939664	4.1108%
Total Power:	96.31686973	

Figure 10: Innovus post-route power report: total 96.32 mW, dominated by switching power in the SA at 200 MHz.

5.3.2 FPGA Backend: Vivado on Zynq UltraScale+

The same RTL is synthesised in Vivado and implemented on a Zynq UltraScale+ MPSoC. The Processing System (PS) hosts a Python control program that loads weights and image into the on-chip BRAMs, asserts a pulse-start signal to the NPU, waits for the done flag, and reads the 128-d embedding back over AXI. Figure 11 shows the resulting block design: the PS-side ZYNQ UltraScale+ MPSoC block is connected to the NPU through an AXI Interconnect and three AXI-GPIO blocks (control signals, address bus, and 128-bit readout bus).

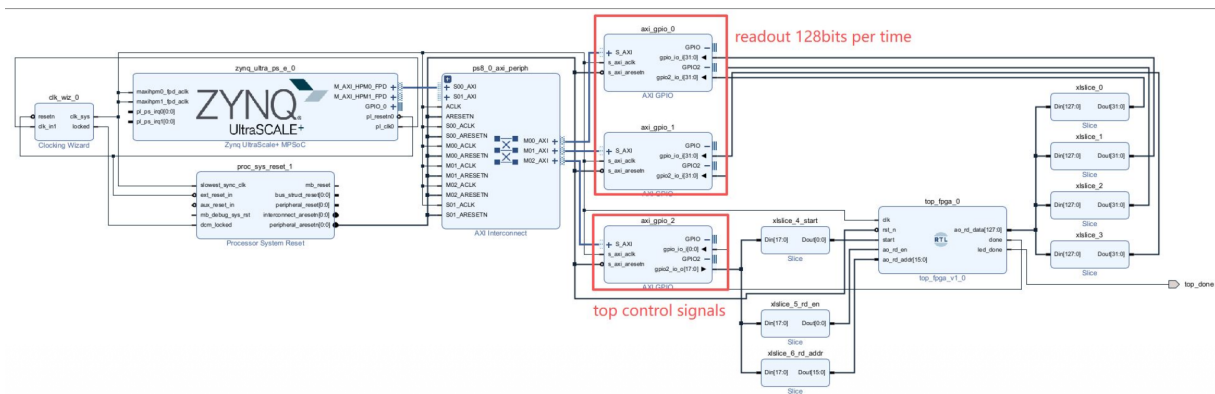


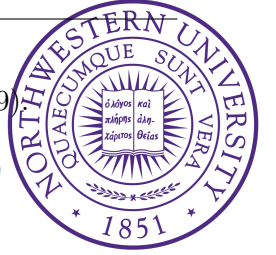
Figure 11: FPGA block design. The PS triggers NPU execution via a control AXI-GPIO; the address bus drives the A/O RAM readout; the readout bus returns 128-bit words to the PS for embedding reconstruction.

Resource utilisation. Vivado implementation reports the resource usage in Table 8. The BRAM utilisation is the dominant resource at 54 %, reflecting the WGT RAM + A/O RAM + Param RAM footprint, while the DSP slice count remains low because most MACs map to LUTs in the SA fabric.

Table 8: FPGA Resource Utilisation (Vivado Implementation)

Resource	Utilisation	Available	Util. %
LUT	107,569	425,280	25.29
LUTRAM	1,308	213,600	0.61
FF	166,393	850,560	19.56
BRAM	583	1,080	53.98
DSP	421	4,272	9.85
IO	1	347	0.29
BUFG	5	696	0.72
MMCM	1	8	12.50

Performance. The bitstream meets timing at 150 MHz and the NPU completes one 112×96 inference in 19,175,557 cycles, i.e. 127.8 ms per image. Compared against a reference INT8 PyTorch run



on an Intel i7-9700 at 3.0 GHz (917.37 ms per image), this is a $\times 7.18$ speed-up (Table 9)

Table 9: FPGA Throughput Comparison (one inference, 112×96 image)

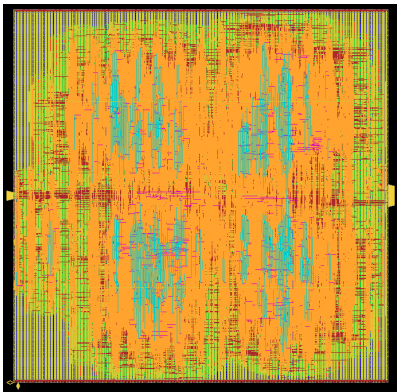
	NPU on Zynq	i7-9700 @ 3.0 GHz
Frequency	150 MHz	3.0 GHz
Total cycles	19,175,557	—
Execution time	127.8 ms	917.37 ms
Speed-up	$\times 7.18$	$\times 1$

A faster RFSoc 4x2 part (PL fabric reaches 500 MHz on the same NPU) would give an estimated $\times 43.08$ speed-up over the same i7-9700 baseline; this remains future work.

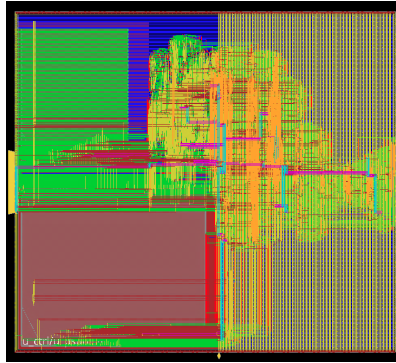
6 Design Optimizations

6.1 ASIC Optimization Journey

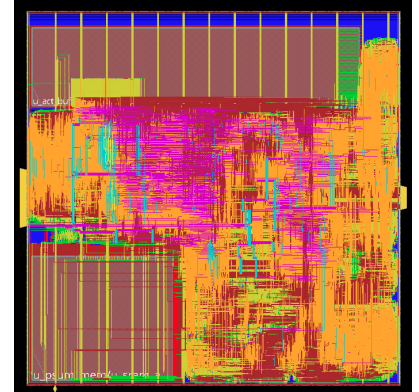
The ASIC implementation went through three architectural generations and four micro-architectural rounds. Each iteration was driven by a measurable shortfall in throughput, power, or area observed at the prior signoff. Figure 12 shows the post-route floorplan for each generation; Table 10 quantifies the metrics behind them.



(a) v1: 1 $\times$ 9-MAC.

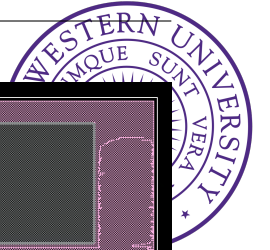


(b) v2: 2 $\times$ 9-MAC + psum SRAM.



(c) v3: 12 $\times$ 12 SA + 2 SRAMs.

Figure 12: ASIC architecture evolution: post-route floorplans for v1, v2, and v3. Visible progression from a single MAC datapath (v1) to a dual-MAC datapath with one OpenRAM macro (v2) to a 12×12 systolic array with two OpenRAM macros (v3, the final design).



(a) v2 detail: psum SRAM macro placed bottom-left.



(b) v3 detail: act_buf and psum SRAM macros bracket the SA.

Figure 13: Macro placement evolution v2 $\rightarrow$ v3.Table 10: ASIC Architecture Evolution: v1 $\rightarrow$ v2 $\rightarrow$ v3

	v1		v2		v3 (final)
Compute	single	9-MAC	dot-	dual 9-MAC (18 mult, 2 out-ch parallel)	hybrid SA ($14 \times 16 + 3 \times 3 + dw 7 \times 7$)
Adder	38-stage ripple carry		40-bit 2-level CLA		CLA + SA pipeline
psum memory	FF array		+ OpenRAM SRAM macro (psum)		+ OpenRAM SRAM macro
act memory	FF array		FF array		+ OpenRAM SRAM macro (A/O buffer)
Cycles / inf	43.16 M		25.77 M		19.18 M
Clock	100 MHz		125 MHz		200 MHz
FPS	2.3		4.85		10.43
Power	26.7 mW		11.4 mW		96.3 mW
Core area	$\sim 285 \text{ K } \mu\text{m}^2$		$479 \text{ K } \mu\text{m}^2$		$518 \text{ K } \mu\text{m}^2$
DRC	0		0		0

v1's single 9-MAC unit produced one output channel per cycle, so 8/9 of the available parallelism was wasted on pointwise layers. v2 doubled the MAC count to 18 (two parallel output channels) and replaced the 38-stage ripple-carry adder with a 40-bit two-level CLA. The biggest win was moving the partial-sum memory from an FF array into an OpenRAM SRAM macro: total cell count dropped and post-route power fell from 26.7 mW to 11.4 mW even though the MAC count had doubled.

v3 replaced the hand-rolled MAC datapath with the hybrid NPU RTL described in Section 5.2: a 14×16 systolic array for 1×1 convolution plus a 3×3 standard array and a $dw 7 \times 7$ engine for the spatial operators. The cycle count drops to 19.18 M and the clock reaches 200 MHz. Power rises to 96.3 mW because the SA fires every cycle at 200 MHz; the meaningful figure of merit is energy per inference, which lands at $9.24 \mu\text{J}$.

Within v3, four cumulative micro-optimizations were applied (Table 11). Each was driven by the previous round's WNS report or power breakdown.

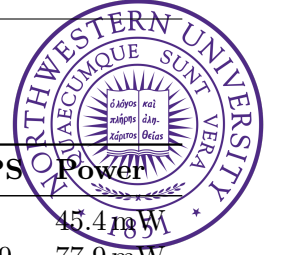


Table 11: v3 Cumulative Micro-Optimizations

Round	Change	F_{max}	FPS	Power
Baseline (2-stage activation)	Stage 0+1 only	87 MHz	6.0	45.4 mW
+ Stage 0+2 split	Lane-select pipelined	150 MHz	10.9	77.9 mW
+ Stage 3 SA <code>row_dot_reg</code>	<code>acc + row_dot</code> split off	174 MHz	12.6	88.2 mW
+ 5.0 ns push	T_{clk} tightened only	200 MHz	14.5	99.1 mW
+ 720×720 area shrink	smaller die, denser placement	200 MHz	14.5	96.3 mW

Two non-architectural fixes were also required to reach signoff. The first is a LEF patch: the OpenRAM-emitted SRAM LEF reserves the whole of metal-3 and metal-4 as obstructions on every macro, but the read-data pins sit on metal-3, so the router has no legal via-3 escape and is forced to cross the obstructions. Without the patch the design reports ~480 SHORT/SPACING DRC violations. The script `fix_sram_lef.py` removes the M3/M4 OBS blocks (the OpenRAM behavioural macros do not actually route on those layers) and snaps off-grid RECT and SIZE coordinates to the 0.005 μm manufacturing grid. After patching, P&R is DRC-clean.

The second is a testbench race. The first 50-layer post-route gate-level simulation reported all 50 layers as mismatches even though RTL was 50/50 MATCH. A `bind`-based probe localised the cause to the testbench: `pixel_in` was driven on `@(posedge clk)` from the address the DUT presented on `x_out`, but the gate-level netlist adds delta-cycle delay on port outputs, so the testbench latched the previous-cycle address. Driving on `@(negedge clk)`, after `x_out` has settled, restored all 50 layers to exact match.

6.2 FPGA Optimization

On the FPGA side the dominant optimizations were resource-targeted rather than algorithmic. The 14×16 SA MAC array is mapped onto LUT-based multipliers rather than DSP slices to keep DSP usage low (9.85%, see Table 8); BRAM placement is biased toward the WGT RAM, which alone accounts for the bulk of the 54% BRAM utilisation. Double-buffered window and weight FIFOs (Buf0/Buf1 in the 3 × 3 engine) hide BRAM read latency behind compute and were necessary to hit the 150 MHz target.

7 Engineering Standards

Table 12 lists the engineering standards and reference specifications used throughout the design.

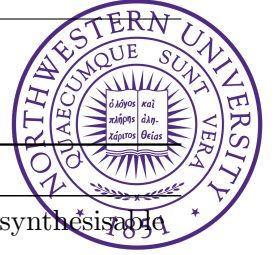


Table 12: Engineering Standards Applied

Domain	Standard	Where used
HDL	IEEE 1800-2017 (SystemVerilog)	All RTL modules (synthesizable subset)
HDL	IEEE 1364-2005 (Verilog 2005)	OpenRAM-emitted SRAM behavioural models
Library exchange	LEF/DEF (Si2 OpenAccess)	Standard-cell + SRAM physical abstracts; Innovus floorplan import/export
Timing models	Liberty (.lib, Synopsys-de-facto)	Cell delays for Genus / Innovus / Xcelium
Delay annotation	IEEE 1497 (SDF)	Post-route SDF backannotation for gate-level signoff
Constraints	Synopsys SDC	<code>constraints.v3.sdc</code> : clock, IO delay, false paths
Reference floats	IEEE 754 binary32	PyTorch baseline + C-model fp32 reference
Fixed-point	INT8 (data, weights), INT32 (accumulator)	internal datapath
Process design kit	NCSU FreePDK45	ASIC standard-cell library + DRC rules
Memory IP	OpenRAM (open-source)	192×64 act_buf, 512×40 psum SRAMs

Compliance notes. The RTL is written in the synthesizable subset of SystemVerilog 2017 (no `assert`, no *dynamic*, no `class`, limited use of `interface`). All cell timing checks (setup/hold/recovery/removal) follow the standard SDF semantics. The LEF patching script `fix_sram_lef.py` preserves the LEF syntax conventions specified in the Si2 LEF/DEF Language Reference; no extension keywords are introduced.

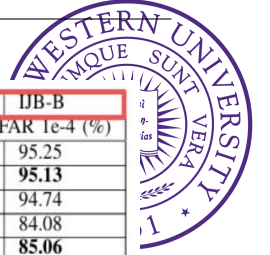
8 Testing / Verification

8.1 Verification Flow

Verification is run in two layers. The model-level layer establishes that the INT8 fixed-point representation preserves face-verification accuracy on standard benchmarks; the hardware-level layer establishes that the RTL is bit-exact against the chosen C-model on every layer.

Model-Level: Face-Verification Benchmarks

The model accuracy reference comes from QuantFace [1], which trains a MobileFaceNet-class network in INT8 (w8a8) with synthetic-data low-bit quantization. We adopt its INT8 baseline directly and check that our hardware-spec fixed-point arithmetic matches the QuantFace PyTorch numbers on the standard face-verification benchmarks LFW, CFP, AgeDB-30, CALFW, CPLFW, IJB-B and IJB-C. QuantFace’s published accuracy table (reproduced in Figure 14) provides the reference per-benchmark numbers across multiple network backbones; the bottom MobileFaceNet (1.1 M params) row at INT8 (w8a8) is the configuration our hardware runs.



Model	param	Quantization data	Bits	Size (MB)	LFW	CFP	AgeDb-30	CALFW	CPLFW	IJB-C	IJB-B
					Accuracy (%)					IAR@FAR 1e-4 (%)	
ResNet100	65.2M	-	FP32	261.22	99.83	98.40	98.33	96.13	93.22	96.50	95.25
		Real data	w8a8	65.31	99.80	98.31	98.13	96.05	92.92	96.38	95.13
		Synthetic data	w8a8	65.31	99.80	98.14	97.95	96.02	92.90	96.09	94.74
		Real data	w6a6	49.01	99.55	89.14	95.85	95.42	85.63	85.80	84.08
		Synthetic data	w6a6	49.01	99.45	91.00	96.43	95.58	86.60	87.00	85.06
ResNet50	43.6M	-	FP32	174.68	99.80	98.01	98.08	96.10	92.43	95.74	94.19
		Real data	w8a8	43.67	99.78	97.70	98.00	96.00	92.17	95.66	94.15
		Synthetic data	w8a8	43.67	99.78	97.43	97.97	95.87	92.08	95.18	93.67
		Real data	w6a6	32.77	99.70	95.00	97.17	95.87	90.17	91.74	90.07
		Synthetic data	w6a6	32.77	99.68	95.17	97.43	95.70	90.38	90.72	89.44
ResNet18	24.0M	-	FP32	96.22	99.67	94.47	97.13	95.70	89.73	93.56	91.64
		Real data	w8a8	24.10	99.63	94.46	97.03	95.72	89.48	93.56	91.57
		Synthetic data	w8a8	24.10	99.55	94.04	97.07	95.58	89.53	92.87	91.01
		Real data	w6a6	18.10	99.52	93.23	96.55	95.58	88.37	93.03	91.08
		Synthetic data	w6a6	18.10	99.55	93.34	96.62	95.32	89.05	92.36	90.38
MobileFaceNet	1.1M	-	FP32	4.21	99.47	91.59	95.62	95.15	87.98	90.88	88.54
		Real data	w8a8	1.10	99.43	91.40	95.47	95.05	87.95	90.57	88.32
		Synthetic data	w8a8	1.10	99.35	90.84	94.37	94.78	87.73	89.21	86.98
		Real data	w6a6	0.79	98.87	87.69	93.03	93.30	84.57	83.13	80.53
		Synthetic data	w6a6	0.79	99.08	87.64	91.77	93.48	84.85	82.94	80.58

Figure 14: QuantFace face-verification benchmark accuracy across LFW, CFP, AgeDB-30, CALFW, CPLFW, and IJB-B/IJB-C. Source: QuantFace [1]. The MobileFaceNet w8a8 row (highlighted) is the configuration adopted by this design.

Two of these benchmarks (LFW and AgeDB-30) are re-run with our hardware-spec fixed-point datapath to confirm the quantisation chosen for hardware does not regress against the QuantFace PyTorch reference. Results are in Table 13: the LFW accuracy difference is 0.20 % and the AgeDB-30 difference is -0.12% , both within the statistical error of the protocol.

Table 13: Hardware-Spec Fixed-Point vs QuantFace PyTorch Reference

	LFW	AgeDB-30
QuantFace PyTorch (w8a8)	99.33 %	95.12 %
HW-spec fixed-point (this work)	99.13 %	95.23 %
Accuracy drop	0.20 %	-0.12%

Accuracy here is measured under the standard verification protocol (image pair labelled 1 if same person, 0 otherwise).

Hardware-Level: Bit-Exact Equivalence

Once the C-model is fixed against the QuantFace numbers, every subsequent stage is verified by direct comparison against the C-model golden hex vectors layer by layer:

PyTorch (INT8) $\rightarrow$ C-model $\rightarrow$ per-layer golden hex $\rightarrow$ implementation under test $\rightarrow$ bit diff.

The C-model is bit-exact against the chosen INT8 arithmetic and is the canonical reference. Table 14 summarises the coverage matrix for both implementation tracks.

Table 14: Verification Coverage Matrix

Level	ASIC	FPGA	Method
RTL functional (50-layer)	✓	✓	Xcelium / Vivado simulator vs C-model
Synthesised netlist (gate-level)	✓	–	Xcelium GL vs C-model golden
Post-route netlist (gate-level)	✓	–	Xcelium GL on Innovus netlist vs C-model
SDF-annotated (timing)	✓	–	Xcelium GL + SDF, check timing violations
On-board	–	✓	Zynq PS readback vs reference embedding
End-to-end embedding	✓	✓	128-d vector vs C-model



8.2 ASIC Verification: Three-Tier + SDF Signoff

The ASIC path is verified at four progressively stricter levels; results are summarised in Table 15.

Table 15: ASIC Verification Results

Level	Stimulus	Delay model	Result
L1	RTL	zero-delay	50 / 50 MATCH
L2	Genus syn netlist	zero-delay	50 / 50 MATCH
L3	Innovus post-route	zero-delay	50 / 50 MATCH
L4	Innovus post-route	SDF annotated	50 / 50 MATCH, 0 violations
End-to-end embedding (RTL, 13 face images): 128 / 128 exact, MaxDiff = 0			

Levels L1–L3 collectively establish that the design behaves identically at the RTL, synthesised, and post-route stages. A formal Logic Equivalence Checker (Conformal LEC, Formality) would normally produce this result in seconds; without one available we substitute a full 50-layer gate-level simulation against the C-model golden hex vectors. The trade-off is wall-clock time. Table 16 summarises how long each of the three gate-level variants took on the same machine.

Table 16: Gate-Level Simulation Wall-Clock Comparison

Run	Netlist	Delay model	Layers done	Wall-clock
<code>make gl</code>	Genus syn	zero-delay	50 / 50	~78 h
<code>make pnr</code>	Innovus post-route	zero-delay	50 / 50	~84 h
<code>make pnr_sdf</code>	Innovus post-route	SDF annotated	50 / 50	~163 h

Post-route simulation runs about $1.1\times$ slower than synthesised because of the added CTS buffers, filler cells, and hold-fix buffers in the post-route netlist. SDF backannotation adds another $1.8\times$ on top: every net carries an RC delay and every flop fires four timing checks (setup, hold, recovery, removal). High-resolution early layers (0–2) dominate all three runs and account for roughly half the wall-clock time.

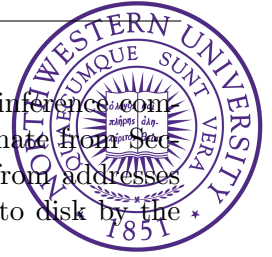
L4 uses the Innovus-exported SDF (`mfn_frontend_top.v3.final.sdf`, 117MB) backannotated into the same gate-level simulation. After 163h of wall-clock time the full 50-layer run completed with all 50 layers bit-exact against the C-model and zero `TCHK_VIOL`, setup-violation, or hold-violation events. In the absence of PrimeTime SI this is the strongest signoff we can produce: the netlist runs MobileFaceNet correctly at 200 MHz with real cell and wire delays and live setup/hold checks.

The verification flow itself had one early failure worth recording. The first gate-level run reported 50/50 mismatches even though RTL was 50/50 MATCH. A SystemVerilog `bind`-based probe traced the cause to a testbench race: `pixel_in` was driven on `@(posedge clk)` from the DUT's `x_out` output, but the gate-level netlist has delta-cycle delay on port outputs, so the testbench was latching the previous-cycle address. Moving the drive to `@(negedge clk)` (after `x_out` has settled) restored all 50 layers. The same testbench is now used for every gate-level and SDF run.

Limitations to keep in mind: this is a single typical-corner signoff (FreePDK45 does not ship multi-corner Liberty), there is no signal-integrity or cross-talk analysis, and the equivalence is shown by simulation rather than formal proof.

8.3 FPGA Verification: Zynq On-Board Readback

The FPGA bitstream is verified end-to-end with hardware in the loop. After loading the PL configuration, the PS-side Python program initialises the NPU (`o1.download()`), asserts the start pulse (`pulse_start()`), polls the done flag (`wait_done(20.0, 0.001)`), and reads back the embedding vector word by word over the 128-bit AXI-GPIO readout bus by iterating addresses `0x40000–0x40007`



and calling `request_read(addr)` followed by `read_128bit_data()`. A representative inference completes in *0.128619 s* of wall-clock time, consistent with the 127.8 ms cycle-budget estimate from Section 5.3.2 after PS round-trip overhead. The 128 8-bit embedding values read back from addresses `0x40000–0x40007` are compared byte by byte against the C-model reference written to disk by the same hex-generation script that drives the ASIC simulation testbench.

Figure 15 shows the reference embedding (left, from the C-model) alongside the PS readout from a real Zynq run (right). The two streams match bit-for-bit on every address.

```

40000 00000000000000000000000000000000
40001 a737e3d480127f1d50f7367f8039ec91
40002 177c7fa3e98080f9c363077f7f12d79f
40003 7f7f80807f1db90a8022ae807f967fa8
40004 5784a88080ce17e543a32a80e55ee0dc
40005 7f9256efa77fde7f80260c071e7f1f80
40006 057f6a7fff087fcc5468365a16bbcdc4
40007 80a619807fef5480c680860ace082afe
40008 4d7f1cc5a67f80717f80d07f747f809a
40009 00000000000000000000000000000000

```

(a) C-model reference embedding.

```

done before start = 0
done, elapsed = 0.128619 s
40000 0xa737e3d480127f1d50f7367f8039ec91
40001 0x177c7fa3e98080f9c363077f7f12d79f
40002 0x7f7f80807f1db90a8022ae807f967fa8
40003 0x5784a88080ce17e543a32a80e55ee0dc
40004 0x7f9256efa77fde7f80260c071e7f1f80
40005 0x57f6a7fff087fcc5468365a16bbcdc4
40006 0x80a619807fef5480c680860ace082afe
40007 0x4d7f1cc5a67f80717f80d07f747f809a

```

(b) PS-side readout of the embedding vector after `wait.done`.

Figure 15: FPGA on-board verification: reference embedding (left) vs PS-side readback (right). Every 128-bit word matches.

The on-board test exercises the full data path that the simulator alone cannot: the AXI Interconnect, the BRAM-mapped WGT / A/O / Param RAMs, the PS-PL handshake on `start/done`, and the readout bus. Together with the ASIC three-tier signoff in Section 8.2, this closes the loop on functional verification of the shared RTL.

9 Ethical Considerations

Face recognition is a sensitive application, and an accelerator that lowers its power and cost is not neutral. Three concerns shape what we built and what we left out: who can be identified, whether the system performs equally across people, and what happens to the biometric data.

Table 17 lists the abuse vectors we considered when choosing the interface. We expose only the 128-d embedding and a match/no-match decision; the raw image is never readable through the chip’s pins. That decision rules out the cheapest forms of mass surveillance (you cannot screenshot people from this hardware) but does not prevent a system integrator from logging the embeddings themselves. The hardware can only constrain the worst options; deployment policy decides the rest.



Table 17: Abuse Vectors and Mitigations

Vector	Risk	Mitigation
Mass surveillance	Cheap edge inference makes installation in public spaces economically trivial.	Hardware cannot self-mitigate. AI Act prohibits real-time remote biometric identification in most public contexts.
Coercive unlock	A phone unlocked by face can be opened by pointing it at the owner.	Out of scope; addressed by the OS via liveness detection.
Template leak	Embedding vectors are permanent; unlike passwords they cannot be revoked once exposed.	Embedding-only output; templates stored in a hardware-isolated enclave in a production SoC.
Presentation attack	A photo, mask, or deepfake can fool an embedding-only pipeline.	Requires a separate liveness stage; this accelerator does not address spoofing.
Model substitution	Replacing the bitstream or netlist could bias the accelerator toward specific identities.	Weights loaded from a signed image in OTP or secure flash.

Demographic bias is a known property of face-recognition networks. NIST FRVT 2019 and Buolamwini and Gebru’s 2018 audit both found higher error rates on darker-skinned and female subjects, and INT8 quantisation does not remove that bias because it is in the trained weights. A deployment built on this accelerator should measure per-demographic error rates on its actual user population, tune the verification threshold to that population, and always offer a non-biometric fallback authentication path.

Privacy compliance is a system-level requirement, not a chip-level one. GDPR Article 9 treats biometric data as special-category personal data, and CCPA has similar provisions. Exposing only the embedding helps but does not satisfy these regulations on its own. The integrator still needs consent capture, purpose limitation, template storage controls, and a documented opt-out path. We treat the chip as necessary but not sufficient.

Putting these together, the deployment patterns we consider acceptable are verification (am I who I claim to be?) rather than identification (who is this stranger?), with user-controlled enrolment, on-device template storage, and an audit trail the user can inspect. Anything broader requires regulatory review that is outside the scope of a hardware project.

10 Conclusions

We delivered a hybrid CNN accelerator for MobileFaceNet inference, implemented along two complementary tracks (Zynq UltraScale+ FPGA and FreePDK45 ASIC) from a single INT8 RTL source. Both tracks were verified against the same C-model golden reference. The deliverables map directly onto the four headline outcomes:

- Designed a hybrid CNN accelerator for MobileFaceNet inference using a 14×16 systolic array, a 3×3 standard array, and a 7×7 depthwise engine, all behind a single Switcher.
- Supported dynamically sized 1×1 , 3×3 , and depthwise convolution operators with INT8 (w8a8) quantisation, PReLU, and residual addition.
- NPU runs at 150 MHz on FPGA ($\times 7.18$ over an i7-9700 baseline) and 200 MHz on ASIC ($\times 9.57$).
- Passes both FPGA on-board verification and Innovus SDF-annotated post-route gate-level verification.

Table 18 consolidates the headline metrics against the original constraints from Section 4.



Table 18: Final Metrics vs Design Constraints

Metric	Constraint	ASIC	FPGA	Met?
Clock frequency	≥ 100 MHz	200 MHz	150 MHz	✓
Throughput	≥ 10 FPS	10.43	7.82	ASIC ✓
Power	< 100 mW	96.3 mW	–	✓
Core area	< 1 mm ²	0.518 mm ²	–	✓
DRC / antenna	0	0 / 0	–	✓
50-layer MATCH	required	RTL+syn+pnr+SDF	on-board	✓
Timing WNS	> 0	+0.089 ns	–	✓
Speed-up vs i7	–	$\times 9.57$	$\times 7.18$	–

Did the design meet spec? Yes for the ASIC track. All hard constraints were met and both soft constraints (power, area) cleared with margin. The FPGA track meets the same functional requirements and reaches 7.82 FPS at 150 MHz; a higher-grade RFSoc-class part would scale this proportionally.

What was not delivered.

- No tape-out: the ASIC is design-kit only.
- No formal LEC: we used full GL simulation instead.
- Single typical-PVT corner; no SI / cross-talk analysis.
- The FPGA bitstream targets a development board, not a productised platform.

Table 19: Suggested Follow-on Work

Priority	Task
High	Port the bitstream to an RFSoc 4x2 part (PL fabric reaches 500 MHz) for an estimated $\times 43.08$ speed-up against the i7-9700 baseline.
High	Formal LEC pass (Conformal / Formality) against RTL: seconds of run time replace days of GL simulation.
Medium	Multi-corner ASIC timing closure (slow / fast / typ) and signal-integrity-aware STA when tools allow.
Medium	Integrate a small DMA / AXI shim and tape out a test chip on an MPW shuttle (FreePDK45 educational; real foundry would substitute its own PDK).
Medium	Replace the combinational weight ROM with an external high-bandwidth weight SRAM plus AXI-Lite write port for runtime model updates.

Future work.

Lessons learned. Verification took longer than RTL design. Two specific items dominated the schedule: the gate-level testbench race that required `bind`-based probes to localise (Section 6.1), and the OpenRAM LEF patching needed to clean up DRC. Both would have been easier with deterministic instrumentation in place from day one.

References

- [1] Fadi Boutros, Naser Damer, and Arjan Kuijper. QuantFace: Towards Lightweight Face Recognition by Synthetic Data Low-bit Quantization. In *2022 26th International Conference on Pattern Recognition (ICPR)*, pages 855–862, Los Alamitos, CA, USA, August 2022. IEEE Computer Society.

- 
- [2] Sheng Chen, Yang Liu, Xiang Gao, and Zhen Han. Mobilefacenets: Efficient cnns for accurate real-time face verification on mobile devices, 2018.
- [3] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1):127–138, 2017.
- [4] Anjith George, Christophe Ecabert, Hatf Otroshi Shahreza, Ketan Kotwal, and Sebastien Marcel. Edgeface: Efficient face recognition model for edge devices, 2024.
- [5] Linjun SUN, Weijun LI, Xin NING, Liping ZHANG, Xiaoli DONG, and Wei HE. Gradient-enhanced softmax for face recognition. *IEICE Transactions on Information and Systems*, E103.D(5):1185–1189, 2020.